DCU
Ollscoil Chathair
Bhaile Átha Cliath
Dublin City University

Purposefully different,

consistently excellent

Welcome to EEN1037: Web Application Development

Introduction to Django Web Framework

Week 3

Joe Mullally

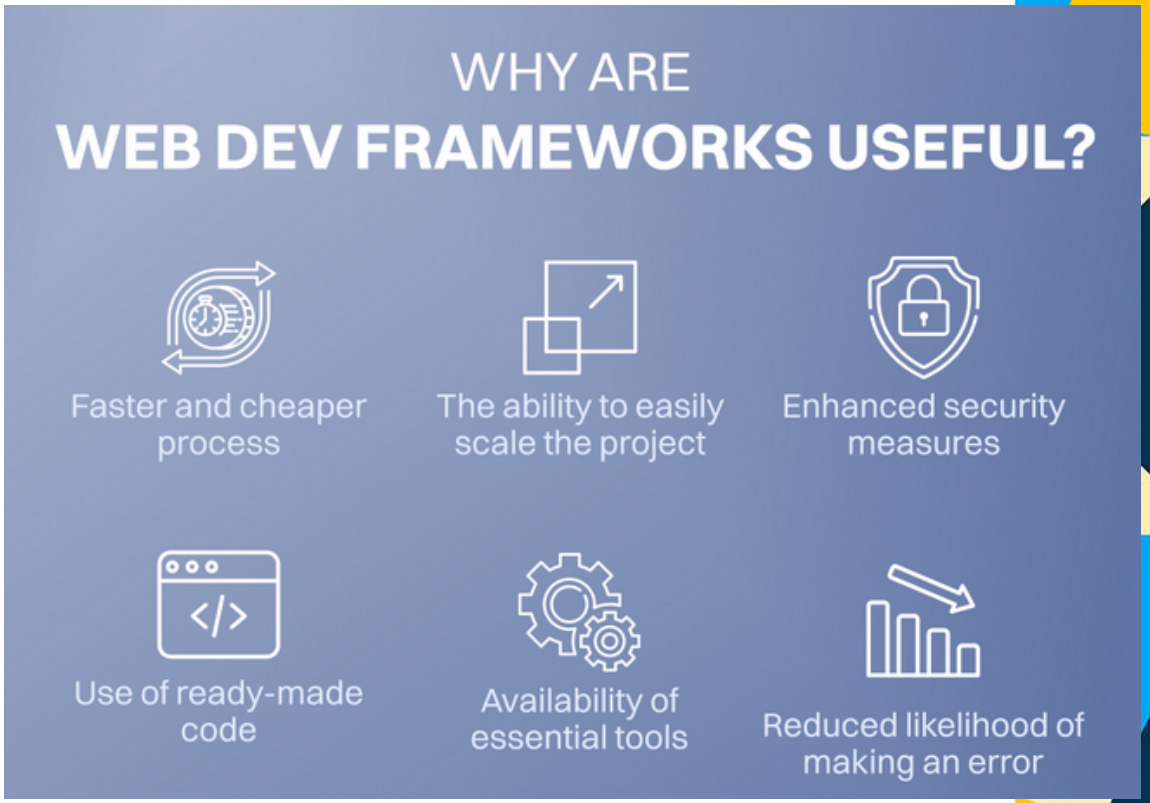
DCU
Ollscoil Chathair
Bhaile Átha Cliath
Dublin City University

Quickstart

- Install Python
  - https://www.python.org/
- Create an empty Django website:
  - Create “requirements.txt” with “Django==6.0.1”
  - python -m venv .venv
  - .venv/bin/pip install Django==6.0.1
  - .venv/bin/django-admin startproject mywebapp
- Go through the tutorials
  - https://code.visualstudio.com/docs/python/tutorial-django
  - https://www.djangoproject.com/start/
  - https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Server-side/Django/Tutorial_local_library_website
  - https://www.w3schools.com/django/index.php

What is a Web Framework?

- Processes incoming HTTP requests and returns responses
- Common reusable code for building dynamic web application services
- Built up over decades of people writing web services
  - Don’t re-invent the wheel!
- “Framework”, you supply:
  - Configuration
  - HTML/CSS/etc
  - Database schema
  - Business logic



Examples of Web Frameworks

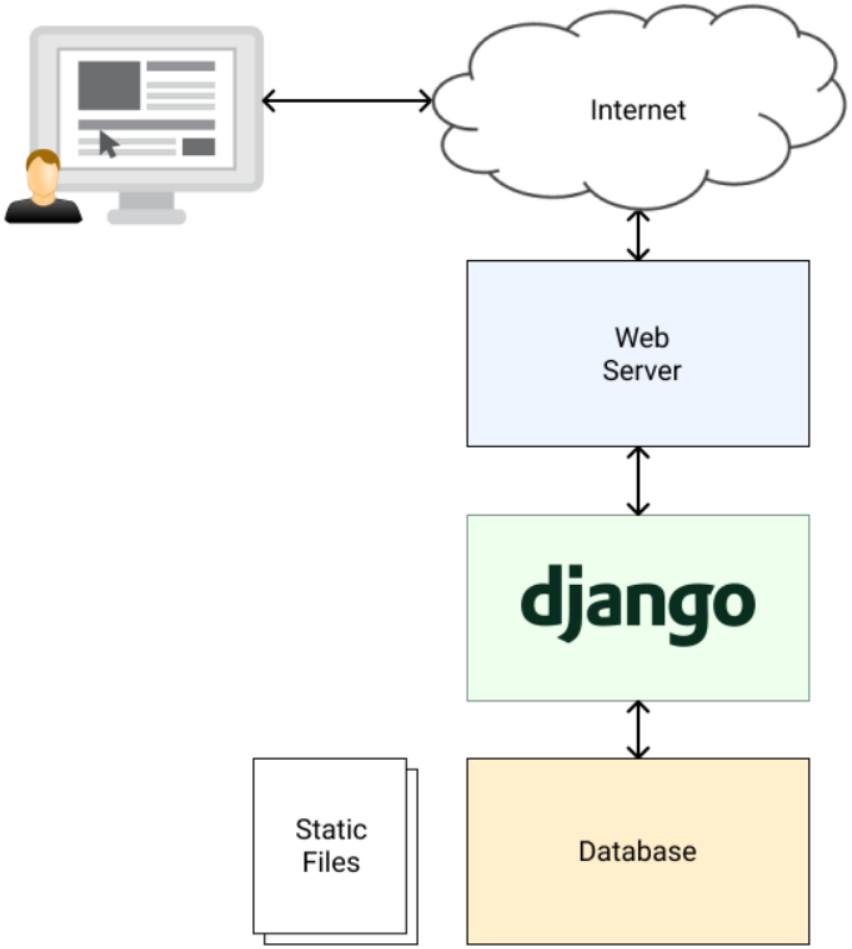
Overview of Python

- Goal: Be easy to learn & read
- Most popular scripting language
- Interpreted, not compiled
  - High-performance possible via C extensions (e.g. NumPy)
  - Just-In-Time (JIT) recently added
- Dynamically Typed
  - Type annotations optional but recommended
  - Lack of types can be drawback in large code bases
- Popular in numerical and scientific communities
  - Graphs & Visualizations
  - Machine Learning
- Many libraries on PyPi, installed with “pip”

```python
# Python program to find the factorial of a number provided by the user.

# change the value for a different result
num = 7

# To take input from the user
#num = int(input("Enter a number: "))

factorial = 1

# check if the number is negative, positive or zero
if num < 0:
   print("Sorry, factorial does not exist for negative numbers")
elif num == 0:
   print("The factorial of 0 is 1")
else:
   for i in range(1,num + 1):
       factorial = factorial*i
   print("The factorial of",num,"is",factorial)
```

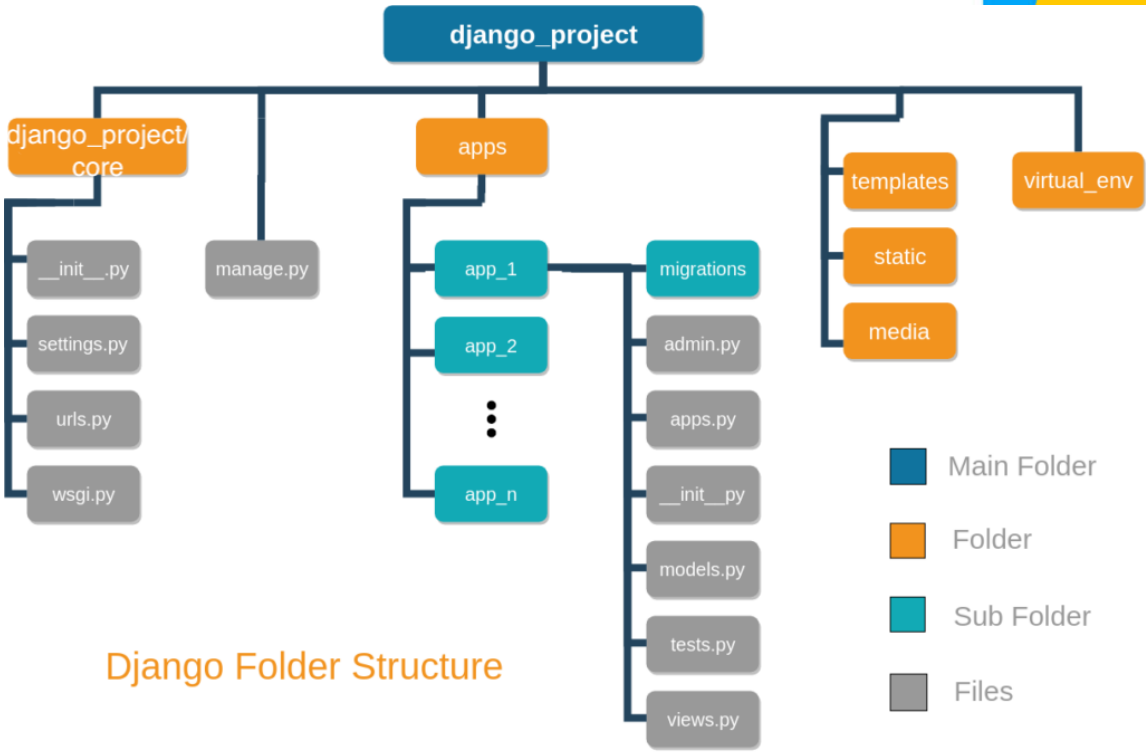
Overview of Django

- Library for Python
- Framework code for handling HTTP requests
- “Batteries-Included” philosophy
  - Contains many standard web framework features
  - Easy to learn
  - Fast to develop production-grade sites
  - Extensible with libraries
- “Model-View-Template” architecture
- Well documented & active community
  - Many videos & tutorials



Django Project Structure

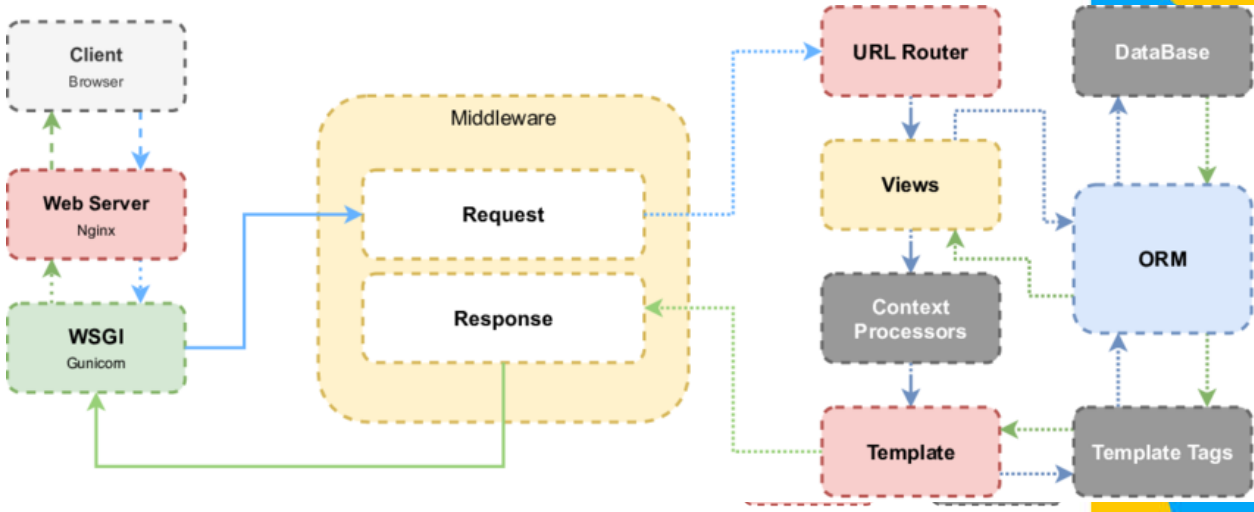
- A Django application consists of:
  - Settings
  - Top level URL routing to “apps”
- One or more apps that contain:
  - Database Model definitions
  - URL routing to individual views
  - View functions
  - HTML Templates
  - HTML, CSS, JS static files
  - Tests, Admin UI settings etc
- When starting off, only one app is recommended.



Django Folder Structure

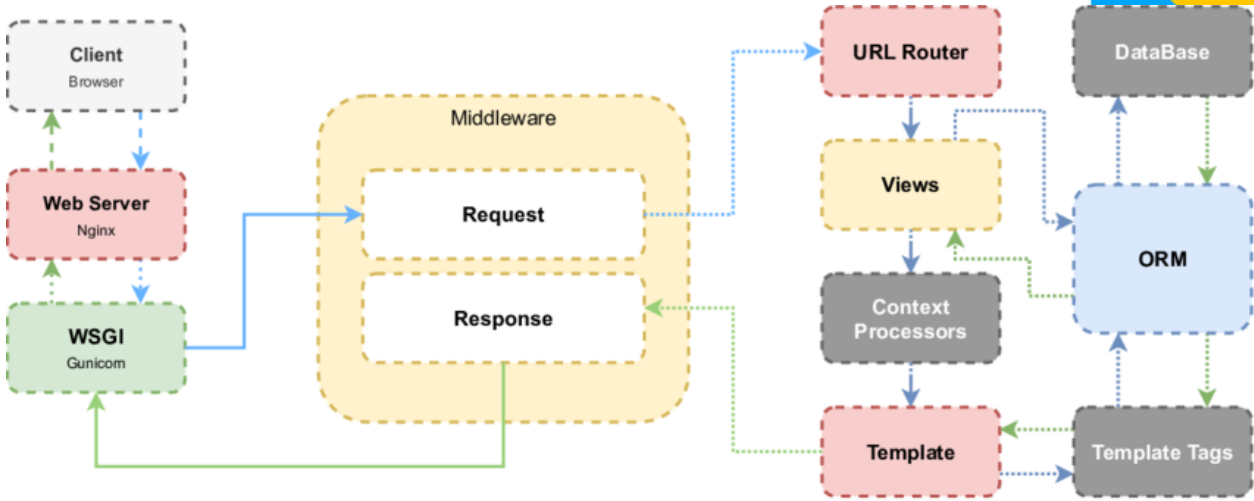
HTTP Request-Response Cycle I

- HTTP Request received by Web Server process
  - django-admin runserver (debug)
  - E.g. Nginx (Production webserver)
- Passed to Django process worker pool
  - E.g. gunicorn, uvicorn
- Django deserializes incoming data into **HttpRequest** object, containing:
  - Method (GET/POST/...)
  - URL (/product-list/235)
  - Query params (?search=cars)
  - Client-side headers
  - Client browser supplied cookies



HTTP Request-Response Cycle II

- HttpRequest passes through **Middleware:**
  - Adds additional info to HttpRequest, e.g. authenticated user
- **URL routing:**
  - Matches URL pattern to specific view function
- **View:**
  - Performs the business logic
- View returns **HttpResponse** object back up the call-stack:
  - Response code
  - Content-Body
  - Content-Type
  - Server set headers
  - Server set cookies
- Web server return serialized **HttpResponse** data



View - “Business Logic”

- Located in “View” functions
- After middleware and URL routing, HttpRequest finally makes it to your code.
- Typically one View per URL
- Look at the incoming request, and decide how to respond:
  - Permission checking
    - Raise HTTP 404 / 403 / 500 exceptions
  - Fetch items from the database
  - Perform additional application logic, e.g. calculating totals for fetched data
  - Render this data using HTML templates
  - OR serialize to JSON
- Return a HttpResponse object when finished

```
def index(request: HttpRequest):
    context = {}
    return render(request, 'myapp/index.html', context)


def product_list(request: HttpRequest):
    products = Product.objects.all()
    context = {
        'products': products
    }
    return render(request, 'myapp/product_list.html', context)


def product_detail(request: HttpRequest, product_id: int):
    product = get_object_or_404(Product, id=product_id)
    cart_quantity = 0
    if request.user.is_authenticated and CartItem.objects.filter(user=reques
        cart_item = CartItem.objects.get(user=request.user, product=product)
        cart_quantity = cart_item.quantity
    initial = {
        'quantity': cart_quantity,
    }
    form = UpdateCartForm(initial=initial)
    context = {
        'product': product,
        'form': form,
    }
    return render(request, 'myapp/product_detail.html', context)
```

HTML Templates

- Django HTML Template syntax
- Expands into well-formed HTML to be sent back to browser
- Control flow logic:
  - {% if … %} ...
  - {% for product in product_list %} …
- Takes “context” object as input
  - Supplied from the View
  - A key-value dictionary containing all variables needed to render the page

```
{% load static %}

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>myapp eShop - {% block title %}{% endblock %}</title>

    <link rel="stylesheet" href="{% static 'myapp/css/style.css' %}">
    {% block extra_style %}{% endblock %}
</head>
<body>
    <nav>
        <ul>
            <li><a href="{% url 'myapp:index' %}">Home</a></li>
            <li><a href="{% url 'myapp:product_list' %}">Products</a></li>
            <li><a href="{% url 'myapp:product_search' %}">Search</a></li>
            {% if user.is_authenticated %}
                <li><a href="{% url 'myapp:cart' %}">Cart</a></li>
                <li><a href="{% url 'myapp:order_history' %}">Orders</a></li>
                <li><a href="{% url 'myapp:profile_edit' %}">Profile</a></li>
```

```
{% extends 'myapp/base.html' %}

{% block title %}Products{% endblock %}

{% block content %}
    <h1>Products</h1>
    <ul>
        {% for product in products %}
        <li>
            {% if product.image %}
                <img src="{{ product.image.url }}" alt="{{ product.name }}" width="50">
            {% endif %}
            <a href="{% url 'myapp:product_detail' product.id %}">{{ product.name }}</a
        </li>
        {% endfor %}
    </ul>
{% endblock %}
```

Example POST request

```python
@staff_member_required
def staff_product_add(request: HttpRequest):
    if request.method == 'POST':
        form = AddProductForm(request.POST, request.FILES)
        if form.is_valid():
            product = Product.objects.create(
                category=form.cleaned_data['category'],
                name=form.cleaned_data['name'],
                description=form.cleaned_data['description'],
                product_code=form.cleaned_data['product_code'],
                image=form.cleaned_data['image'],
                price=form.cleaned_data['price'],
                stock=0,
            )
            messages.success(request, 'Product added successfully.')
            return redirect('myapp:product_detail', product_id=product.id)
    else:
        form = AddProductForm()
    context = {
        'form': form,
    }
    return render(request, 'myapp/staff_product_add.html', context)
```

```html
{% extends 'myapp/base.html' %}

{% block title %}Add Product{% endblock %}

{% block content %}
    <h1>Add Product</h1>
    <form method="post" enctype="multipart/form-data">
        {% csrf_token %}
        <div class="form-group">
            {{ form.category.label_tag }}
            {{ form.category }}
        </div>
        <div class="form-group">
            {{ form.name.label_tag }}
            {{ form.name }}
        </div>
        <div class="form-group">
            {{ form.description.label_tag }}
            {{ form.description }}
        </div>
        <div class="form-group">
            {{ form.product_code.label_tag }}
            {{ form.product_code }}
        </div>
        <div class="form-group">
            {{ form.image.label_tag }}
            {{ form.image }}
        </div>
        <div class="form-group">
            {{ form.price.label_tag }}
            {{ form.price }}
        </div>
        <button type="submit">Add</button>
    </form>
{% endblock %}
```

Form helpers

- Form classes
  - Parses POST request fields
  - Safely validate incoming data – important!

```
# myapp/templates/myapp/author_create.html

  <h1>Create a new Author</h1>
  <form action="{% url 'myapp:author_create' %}" method="post">
    {% csrf_token %}
    {{ form.non_field_errors }}
    <div>
      <label for="id_name">Name:</label>
      {{ form.name }}
      {{ form.name.errors }}
    </div>
    <div>
      <label for="id_content">Date of Birth:</label>
      {{ form.birthday }}
      {{ form.birthday.errors }}
    </div>
    <div>
      <label for="id_content">Nationality:</label>
      {{ form.nationality }}
      {{ form.nationality.errors }}
    </div>
    <input type="submit" value="Submit">
  </form>
```

```
# forms.py

class NewAuthorForm(forms.Form):
    name = forms.CharField(
        max_length=100,
        widget=forms.TextInput(attrs={'class': 'myfieldclass'}),
        label='Title')
    birthday = forms.DateField(
        widget=forms.DateInput(attrs={'class': 'myfieldclass',
               'type': 'date'}),
        label='Date of Birth')
    nationality = forms.CharField(
        max_length=100,
        widget=forms.TextInput(attrs={'class': 'myfieldclass',
               'placeholder': 'Enter Nationality'}),
        label='Nationality'
    )
```

```
# myapp/views.py

def author_create(request: HttpRequest):
    if request.method == 'POST':
        form = NewAuthorForm(request.POST)
        if form.is_valid():
            new_author = Author.objects.create(
                name = form.cleaned_data['name'],
                birthday = form.cleaned_data['birthday'],
                nationality = form.cleaned_data['nationality'],
            )
            return redirect('myapp:author_list')
    else:
        form = NewAuthorForm()
    context = {
        'form': form
    }
    return render(request, 'myapp/author_create.html', context)
```

Database & Object-Relational Mapping

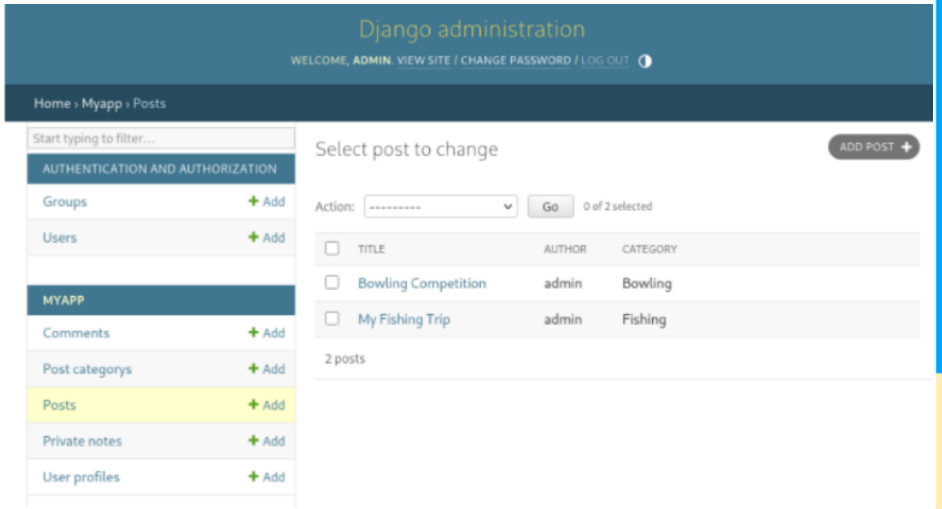
- Built-in ORM
  - Very easy to use
- Define SQL Database tables using Django “Model” classes
  - What data you store depends on your application
- In Application logic, use methods on Model “querysets” to fetch/store data from DB tables
  - Each returned object corresponds to one row in database
  - Under the hood, these methods translate into safe SQL queries that get sent to the DB
  - Supports Create, Read, Update, Delete
- Automatic SQL migration generation for initial creation & changes!
  - Auto-generated migrations may need extra steps or adjustments for complex changes, e.g. moving data from one table to another

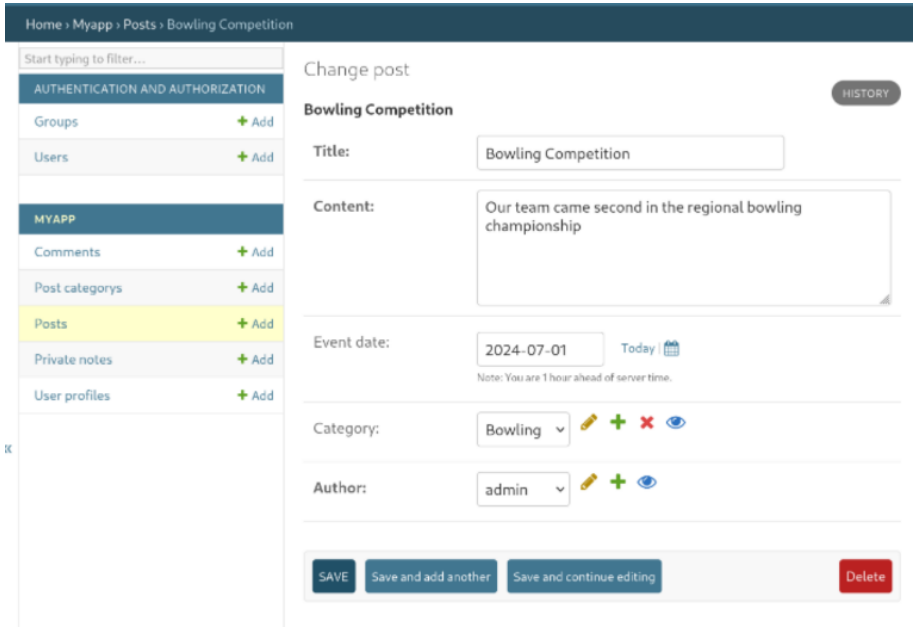
```python
class BookReview(models.Model):
    book = models.ForeignKey(Book, on_delete=models.CASCADE)
    commenter = models.ForeignKey(User, on_delete=models.CASCADE)
    comment = models.TextField()
    time_published = models.DateTimeField()
    rating = models.IntegerField()
    thumbs_up = models.IntegerField()

    def __str__(self):
        return f"@{self.commenter.username}'s review on {self.book.title}"


class UserProfile(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE)
    interests = models.TextField()
    countrycode = models.CharField(max_length=2)

    def __str__(self):
        return f"{self.user.username}'s Profile"
```

```python
# Post.objects.filter(category__name='Fishing').order_by('-created_at').first()

SELECT
        "myapp_post"."id",
        "myapp_post"."title",
        "myapp_post"."content",
        "myapp_post"."event_date",
        "myapp_post"."category_id",
        "myapp_post"."author_id",
        "myapp_post"."created_at",
        "myapp_post"."updated_at"
FROM "myapp_post"
INNER JOIN "myapp_postcategory"
        ON ("myapp_post"."category_id" = "myapp_postcategory"."id")
WHERE "myapp_postcategory"."name" = 'Fishing'
ORDER BY "myapp_post"."created_at"
DESC LIMIT 1
```

Admin UI

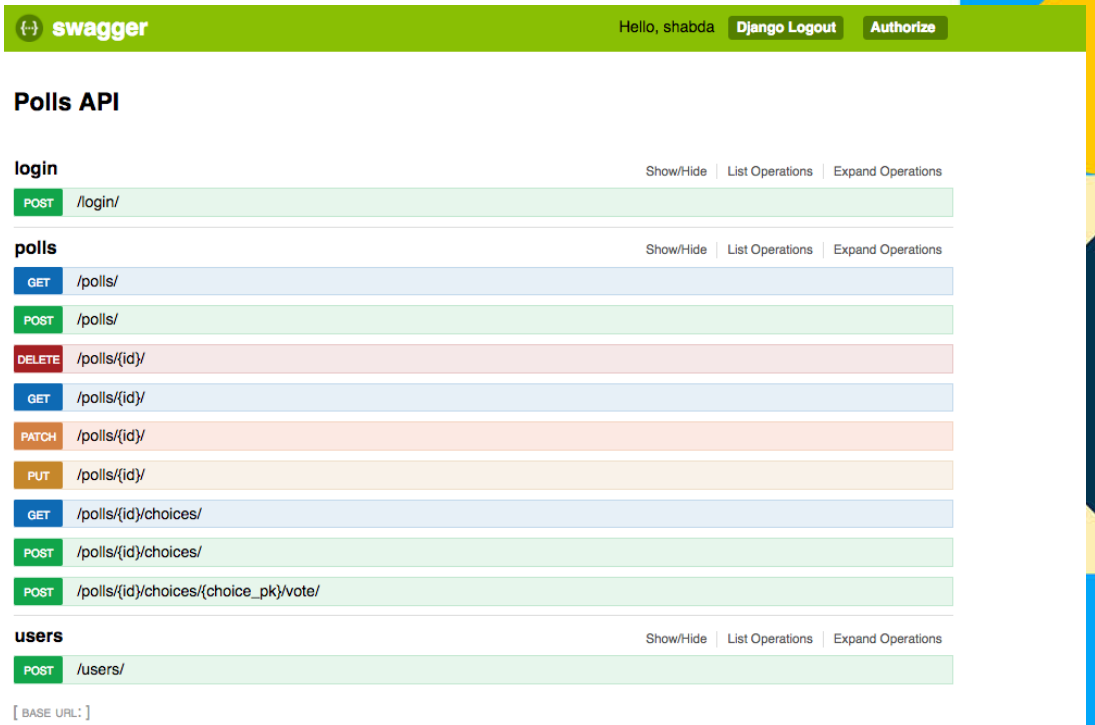
- Easy Web Admin interface for your models
- For internal staff only
  - Don’t expose to customers!
- Can Create/Read/Update/Delete, database items, similarly to code
- Very useful during early development phase:
  - Step 1: Define your models.py
  - Step 2: Test your data model through Admin UI





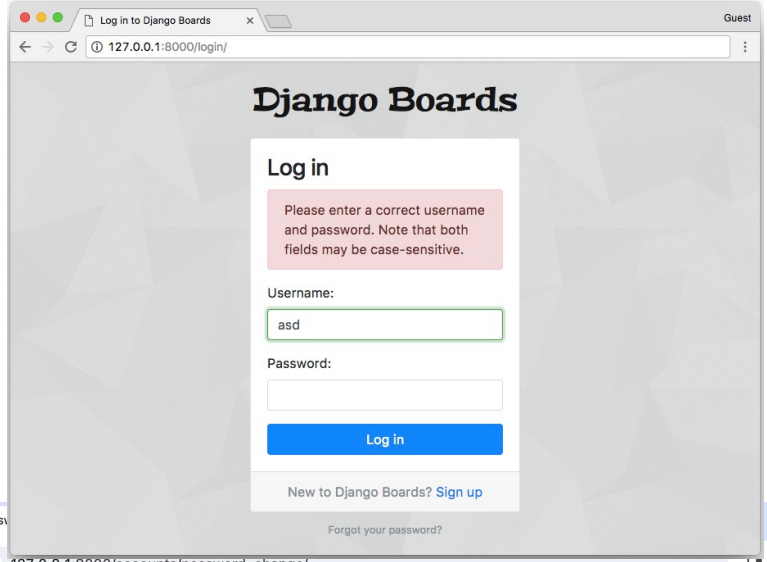
REST APIs

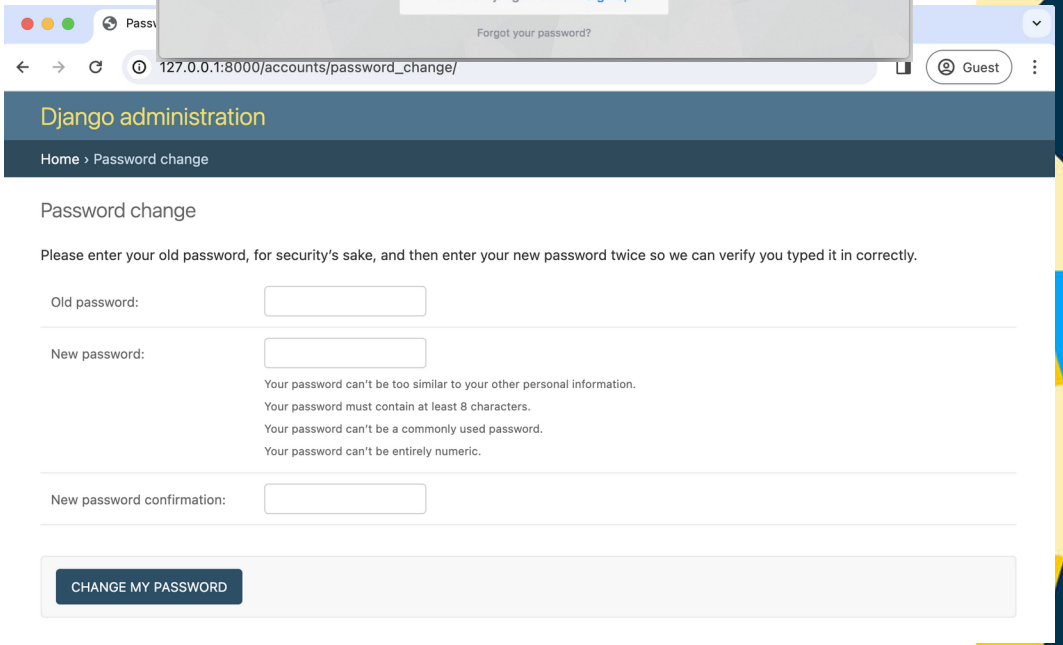
- Can implement REST standard APIs using regular views + JSON/XML serialization
  - But very tedious for every Method/URL combination
- “Django-Rest-Framework” plugin allows for easy auto-generation of well-formed REST URL structure
- Can reflect Model/Database schema directly
  - Warning: Couples customer-facing API structure to Database schema!
  - Or just define manual serializer classes
- Automatic Swagger / OpenAPI 3.0 generation with nice customer-facing API UI
- GraphQL and other API formats available using other plugins



User Authentication

- Don’t invent your own security!
- Includes built-in secure user management:
  - Registration
  - Login
  - Change password
- Can manage users through Admin UI
- Login session tracking using Cookies
  - Can set to expire after certain time
- Middleware sets “request.user”
- Just handles “authentication”, you still need to implement “authorization” for each view
- External authentication providers using plugins, e.g. OIDC, django-allauth, django-keycloak

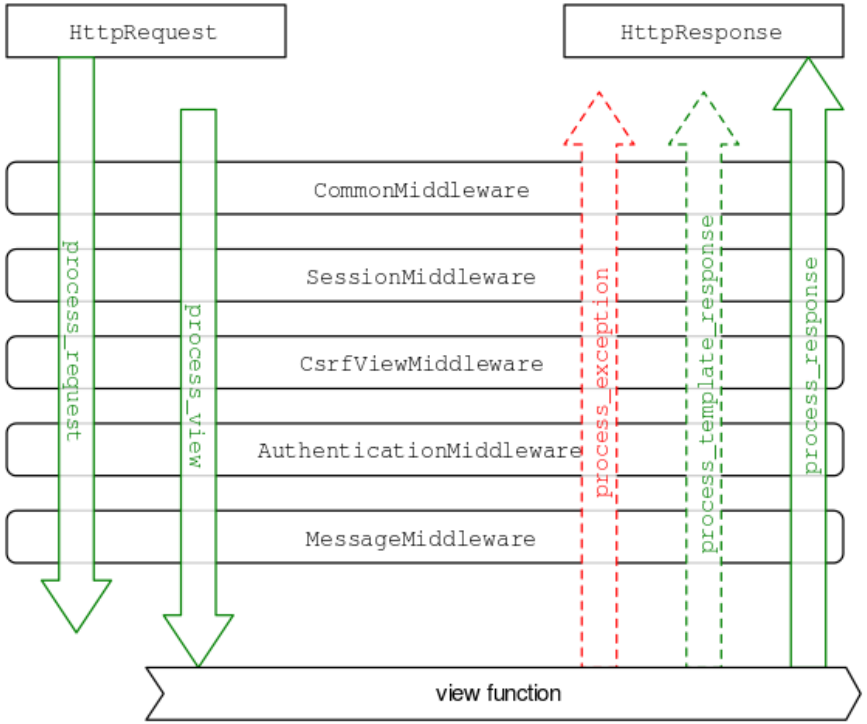




Middleware

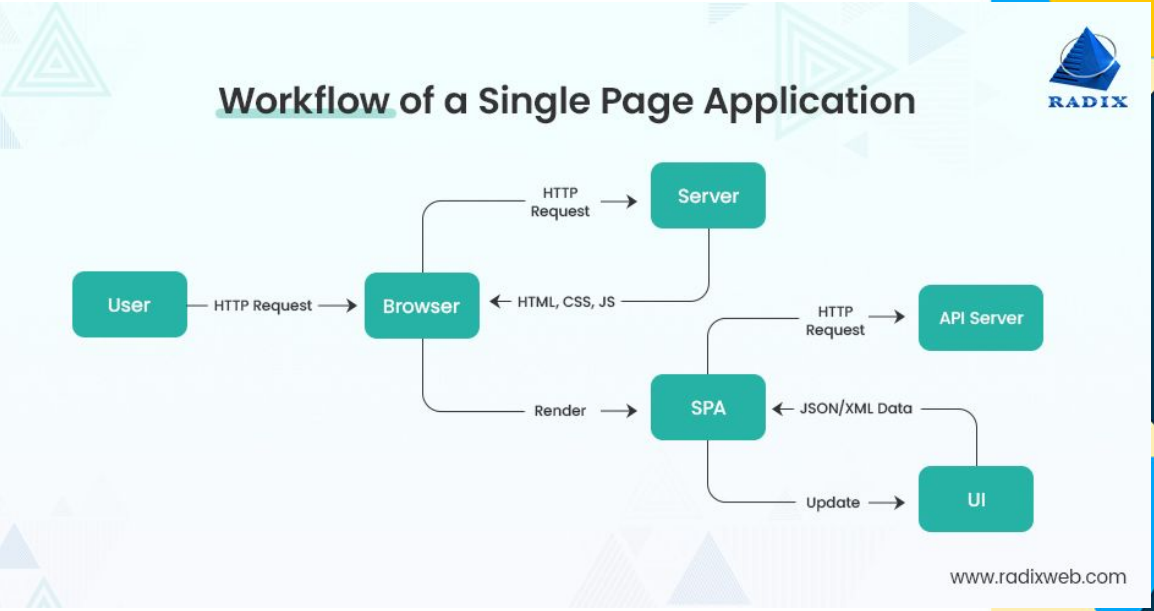
- Starter project provides default middlware:
  - Authentication
  - Session management
  - CSRF protection
  - Status Message injection

```
MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    "django.contrib.sessions.middleware.SessionMiddleware",
    "django.middleware.common.CommonMiddleware",
    "django.middleware.csrf.CsrfViewMiddleware",
    "django.contrib.auth.middleware.AuthenticationMiddleware",
    "django.contrib.messages.middleware.MessageMiddleware",
    "django.middleware.clickjacking.XFrameOptionsMiddleware",
]
```



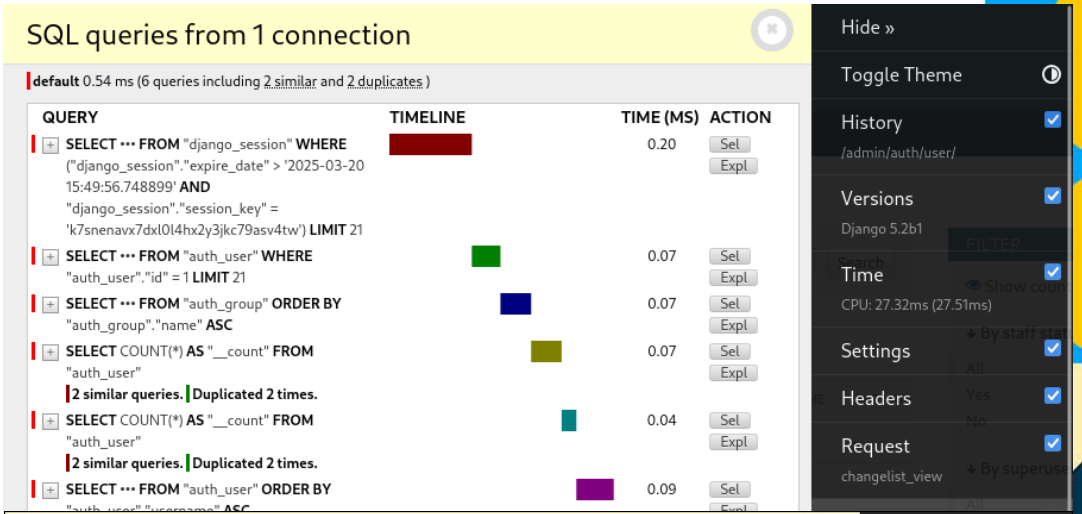
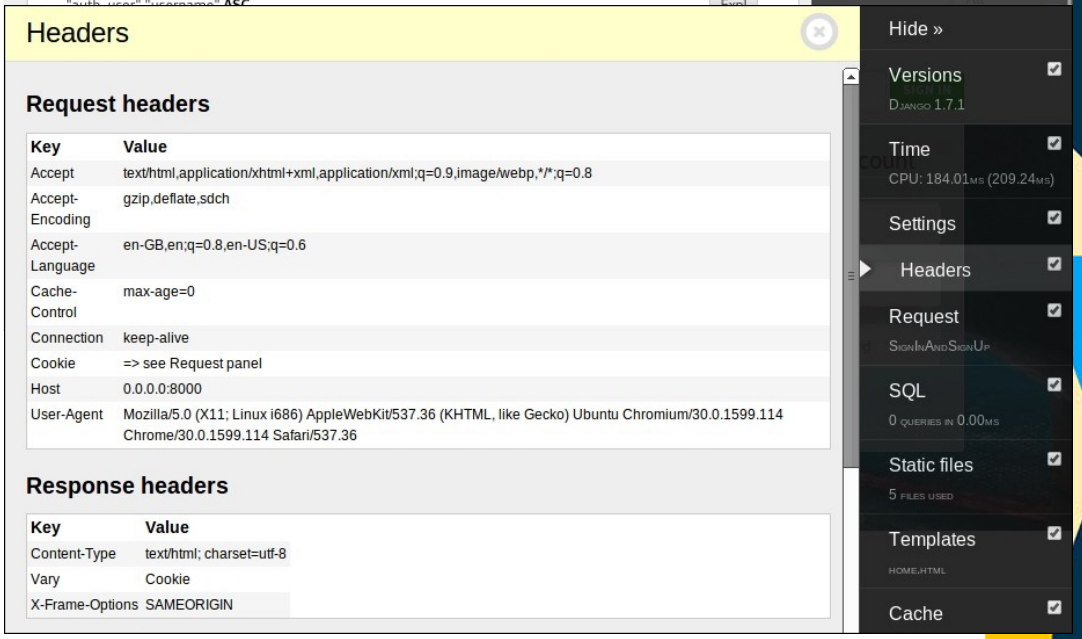
HTML Templates vs SPA + REST?

- Design decision: Use built-in HTML templates or SPAs & REST APIs
- Depends on application
- Modern interactive apps will benefit from SPA
  - => Just use Django for REST / Ajax
- Internal tools benefit from simplicity and security of plain HTML templates
-



Unit Tests

- Can write unit tests for each URL
- Use “client” object to perform specific “requests” against your applications URL endpoints
  - Login as a certain user
  - Supply parameters
  - Perform POST requests
  - Check responses contain expected results & database has been updated as we expect#
- Software Engineering = 100% of your code is covered by automated tests
  - Makes sure existing functionality continues to work as expected while development continues.
- Test-Driven-Development if you write tests first!

Debug Tools

- Standard application logging
  - Default Request Logs
  - Custom Application logs
    - logger.warning
    - logger.info
    - logger.error
  - Database SQL logs
- Django-debug-toolbar shows more detailed information
- Visual Studio Code breakpoints